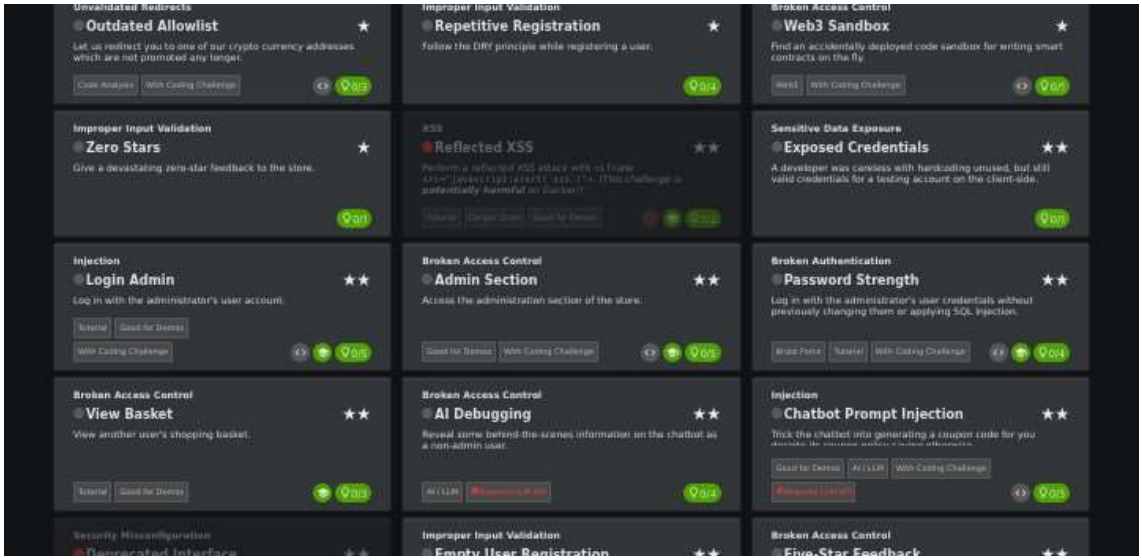
WEEK 8 | ADVANCED VAPT

Attack Chain & Full Exploitation Project

Target: OWASP Juice Shop (local lab environment)

Tools: Kali Linux • Burp Suite Community Edition • Web browser



Prepared for: DG Interns Hub — Cyber Security Internship

Reporting note: This report documents the visible lab workflow and evidence in the supplied screenshots. Findings are treated as observations or hypotheses unless the screenshots prove successful exploitation.

1. Executive Summary

This Week 8 exercise focuses on moving from isolated checks to a defensible attack-chain assessment. The supplied evidence shows a local OWASP Juice Shop instance, normal user-facing flows, and Burp Suite traffic inspection. Requests include product search and application-configuration paths, while responses include product data returned as JSON.

The available screenshots establish that the application was reachable and that HTTP requests could be observed and reviewed in Burp Suite. They do not, by themselves, prove an authentication bypass, an IDOR, successful administrator access, or sensitive-data extraction. Those outcomes should only be reported after a repeatable test with clear before/after evidence.

Assessment goals

- Map key entry points and the main user journey.
- Inspect browser-generated requests and responses using Burp Suite Proxy and Repeater.
- Identify candidate logic and access-control tests for a controlled local lab.
- Explain realistic business impact and recommend layered remediation.

Scope and safety

The target is the locally hosted OWASP Juice Shop at localhost:3000, as shown in the evidence. Testing should remain limited to this intentionally vulnerable lab. Public websites appearing in incidental browser traffic are not treated as targets or evidence for this report.

2. Target, Scope & Methodology

Target application

OWASP Juice Shop is a deliberately vulnerable web application used for security learning. The screenshots show its login and registration pages, product catalogue, search results, product review panel, and basket flow.

In-scope components

- Web UI: login, registration, product catalogue, search, product details/reviews, basket.
- HTTP API paths observed in the local application, including product-search and challenge-related endpoints.
- Burp Suite Proxy history and Repeater views used to inspect requests and responses.

Out of scope

- Any public third-party site or unrelated network destination.
- Destructive actions, real-user data, denial-of-service testing, or persistence.
- Claims of exploit success without repeatable proof.

Method

1) Browse the lab normally. 2) Capture the generated HTTP traffic in Proxy history. 3) Select a request and review its method, path, headers, cookies, and response. 4) Send suitable lab requests to Repeater. 5) Change one parameter at a time and compare status codes, response bodies, and application behavior. 6) Record impact, confidence, and remediation.

3. Attack Surface Mapping

The observed interface exposes several candidate entry points. A complete map should include both UI routes and the corresponding API calls, then identify which routes require authentication and which data is user-specific.

Observed entry points

- Authentication: login form, registration form, password-reset link.
- Product browsing: catalogue, product search, product detail and review display.
- Shopping flow: add-to-basket controls, basket quantity controls, checkout entry point.
- API traffic: product-search request and JSON product response visible in Burp.
- Administrative paths: an application-configuration path was requested; the screenshot alone does not establishwhether access was authorized or denied.

Recommended mapping fields

For each route, record HTTP method, path, parameters, cookies/session context, authentication requirement, expected role, response status, data returned, and whether the result changes when logged out or logged in as a normal user.

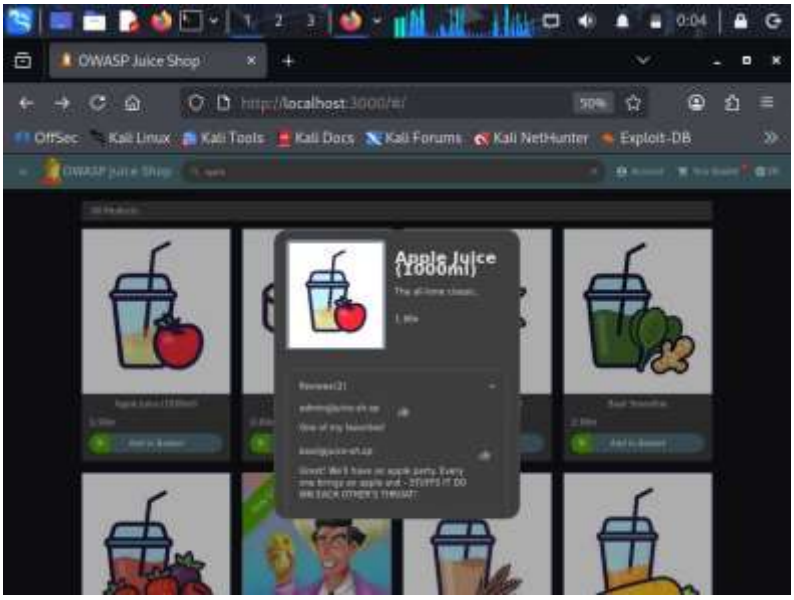


Figure 1. Product catalogue view used to identify browsing and basket-related entry points.

4. Authentication & Session Review

The supplied screenshots show the login and registration interfaces and a test-account login form populated with a sample email. The password is masked in the image. This is evidence of the user interface, not proof of weak authentication.

Controlled checks to perform

- Verify that protected pages and APIs reject requests without a valid session.

- Compare the same request when logged out and after a normal test-user login.
- Check whether logout invalidates the session and whether old session cookies still work.
- Review cookie attributes such as HttpOnly, Secure (where HTTPS is used), and SameSite.
- Test password policy and account recovery only using accounts created for the lab.

Evidence standard

Record the exact request and response for each test. A successful authentication weakness requires a reproducible difference—such as protected data being returned without a valid session—not merely a visible login form or a masked password field.

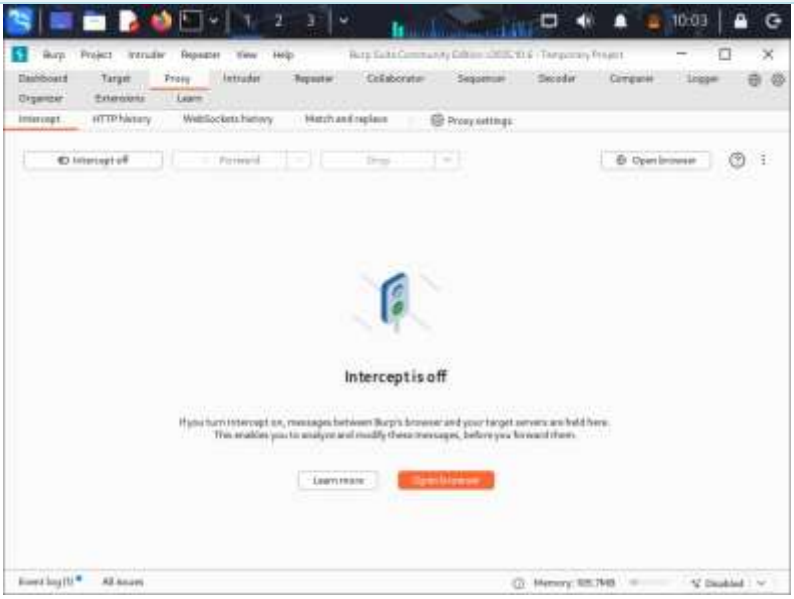


Figure 2. Juice Shop login page observed during the lab workflow.

5. Burp Suite Proxy: Traffic Inspection

Burp Suite Proxy provides a record of browser-to-application HTTP traffic. The screenshots show the Proxy workspace, HTTP history, and request/response inspection panels. This supports traffic analysis and request selection for further testing.

Review checklist

- Confirm the host is the intended local lab (localhost:3000).
- Inspect the method and path before editing a request.
- Review query parameters, cookies, and headers for values that may affect authorization or business logic.
- Compare response status, headers, content type, and response body.
- Avoid treating incidental requests to unrelated external domains as target findings.

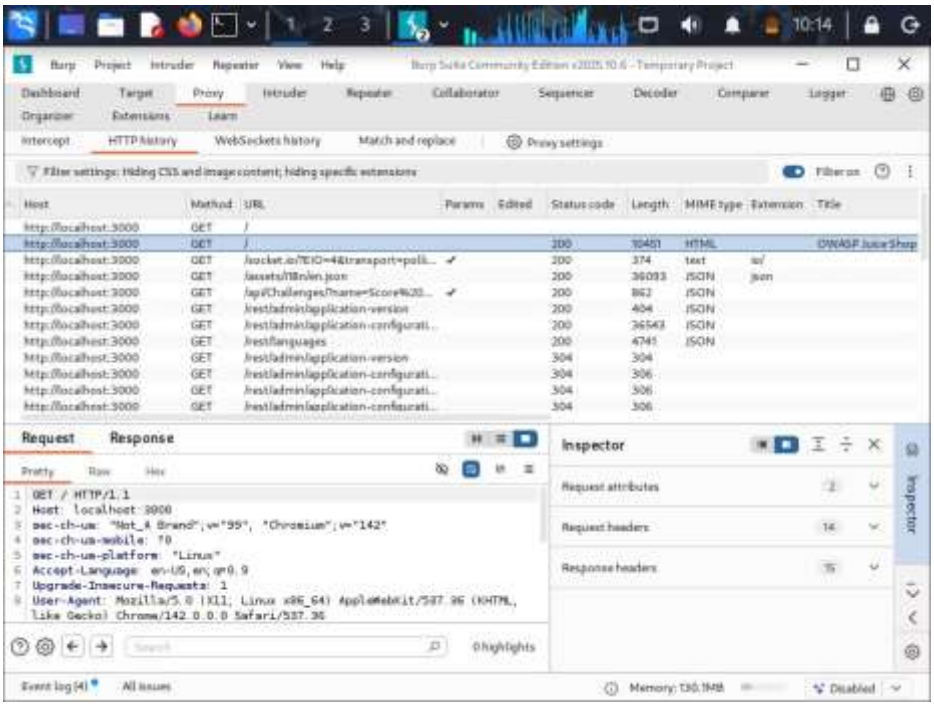


Figure 3. Burp Suite HTTP history showing local application requests.

6. Product Search API: Request & Response

The evidence includes a Repeater request to a product-search route with a query parameter and a JSON response containing product records. This confirms that the product-search API can be inspected directly and that its response exposes catalogue fields such as product identifiers, names, descriptions, prices, and image metadata.

What to validate

- Input handling: empty, long, encoded, and unexpected search values.
- Response consistency: valid JSON, appropriate status codes, and safe error messages.
- Data minimization: only fields needed by the public catalogue should be returned.
- Authorization: if a route is intended to return private data, verify access checks server-side.

Interpretation

Public product catalogue information is normally expected to be available to unauthenticated users, so its presence is not automatically a vulnerability. Any finding must identify an unexpected disclosure, unsafe behavior, or measurable business impact.

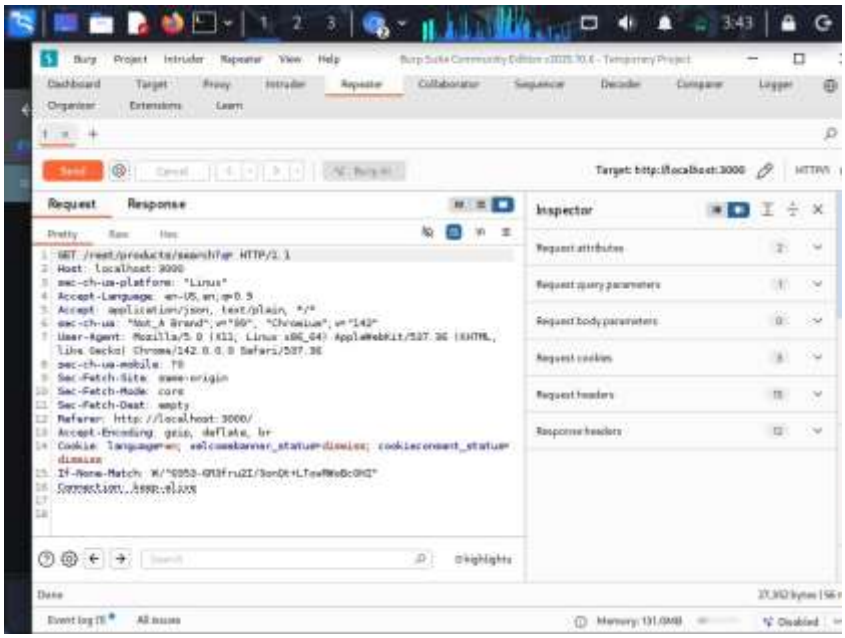


Figure 4. Product-search request reviewed in Burp Suite Repeater.

7. Response Analysis & Parameter Tampering

A response screenshot shows a successful HTTP response and structured JSON product data. A separate request view shows an application-configuration path. These are useful starting points for controlled validation, but response content and access-control results must be captured together before assigning a finding.

Safe parameter-tampering workflow

- Send the original request first and save the baseline response.
- Change only one non-destructive parameter at a time.
- Compare status code, response length, returned fields, and visible application behavior.
- Repeat with a logged-out session and a normal lab account where appropriate.
- Restore the original value and document whether the behavior is repeatable.

Administrative endpoint caution

The presence of a request to an administrative-looking route does not prove that an attacker can access administrator data. Confirm the response body and test with appropriate role/session comparisons. If the request is blocked, record the denial as positive security behavior.

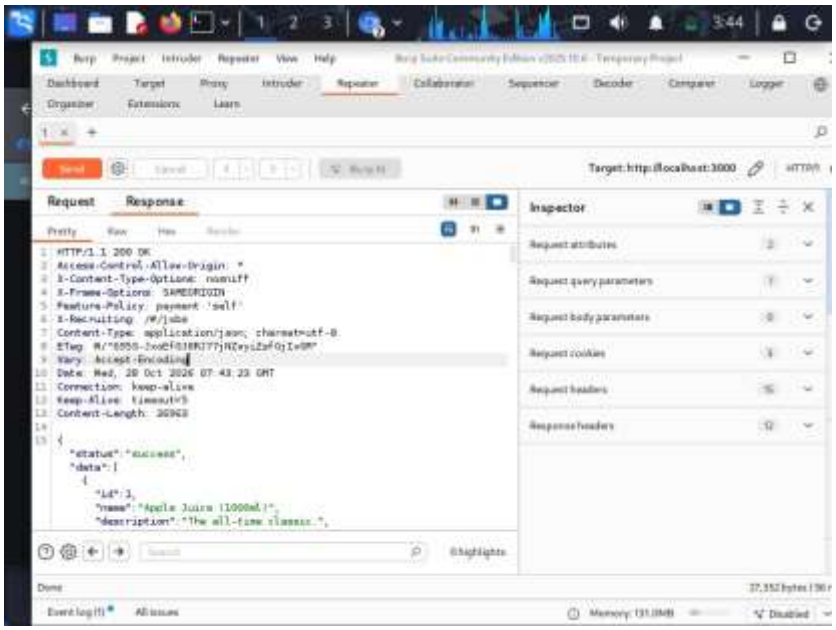


Figure 5. JSON response to a product-search request.

8. Attack-Chain Hypothesis & Validation Plan

The assignment asks for a chain that combines multiple weaknesses. Based on the current evidence, the following is a validation plan—not a claim that a complete exploit was achieved.

Candidate chain

Step 1 — Entry point: map login, search, product, basket, and relevant API routes. Capture a baseline request in Burp.

Step 2 — Logic / parameter test: vary a harmless search or basket parameter and check whether the server validates it and applies the expected business rule.

Step 3 — Access-control comparison: choose a user-specific resource in the lab, if available, and compare responses between logged-out and authenticated test sessions. Do not use real-user identifiers or data.

Step 4 — Impact confirmation: if a control fails, demonstrate only the minimum proof needed (for example, one test record), then stop and preserve the request/response evidence.

Step 5 — Break the chain: repeat the same requests after remediation or with expected authorization controls in place.

Required proof before reporting a confirmed chain

- Each step has an exact request and response.
- The result is repeatable and tied to the intended local target.
- The impact is demonstrated without exposing unrelated data.
- The chain fails when the relevant control is applied.

9. Risk & Business Impact Analysis

Severity should be assigned only after exploitability and impact are demonstrated. The table below is a preliminary risk guide for the candidate areas, not a claim that each issue was found.

Candidate area	Potential impact if confirmed
Authentication/session weakness	Unauthorized account access or session misuse
Broken access control / IDOR	Access to another user's data or actions
Business-logic flaw	Unauthorized checkout, price/quantity manipulation, or workflow
Excessive API data	Unnecessary exposure of internal or sensitive fields
Input validation issue	Unexpected behavior or injection risk

Business framing

A successful chain can turn a small validation or authorization weakness into a larger compromise. For a shopping application, realistic outcomes may include viewing another user's order, changing an unauthorized value, or accessing administrative data. The exact impact must be tied to observed evidence and the target's intended behavior.

10. Remediation Strategy

Apply fixes at the server-side control layer. Client-side validation improves usability but cannot replace server-side authorization and business-rule enforcement.

Authentication and sessions

- Use established authentication libraries and secure password hashing.
- Rotate session identifiers after login or privilege changes; invalidate sessions on logout.
- Set appropriate cookie attributes and apply CSRF protections where relevant.
- Rate-limit authentication and recovery flows and avoid revealing whether an account exists.

Authorization and IDOR

- Authorize every request on the server using the authenticated user and required role.
- Scope database queries to the current user's permitted records.
- Do not rely on hidden buttons, client-side route guards, or unpredictable identifiers alone.

Business logic and APIs

- Recalculate prices, totals, permissions, and workflow state on the server.
- Allow-list accepted fields and validate type, range, and ownership.
- Return only the fields needed by the client; use consistent error handling.
- Add regression tests for the exact request/response pairs used during testing.

Breaking the chain

The most reliable remediation is layered: enforce authentication, verify object-level authorization, validate business state on the server, minimize API responses, and monitor repeated rejected requests. Fixing only one client-side screen is insufficient if the same API remains directly callable.

11. Evidence Gallery & Conclusion

The following images document the lab setup and request-inspection workflow. Use each screenshot with a short caption and the corresponding test ID in the final submission.

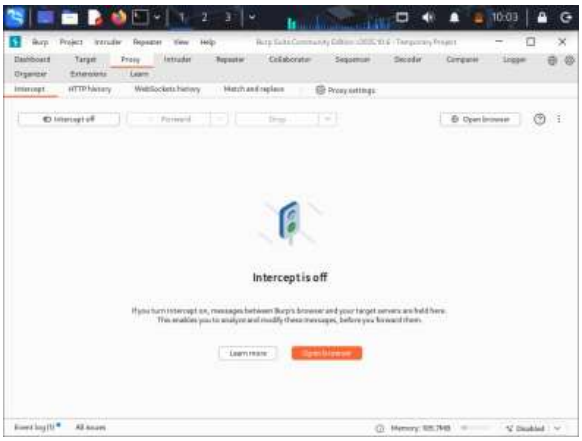


Figure: Login interface / authentication entry point.

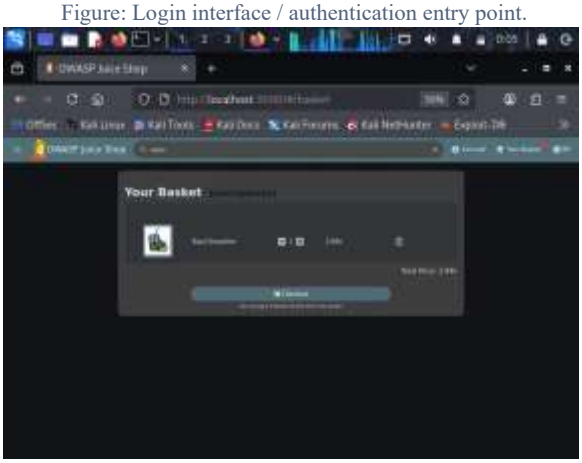


Figure: Search results page / product discovery.



Figure: Burp Suite Repeater request workspace.

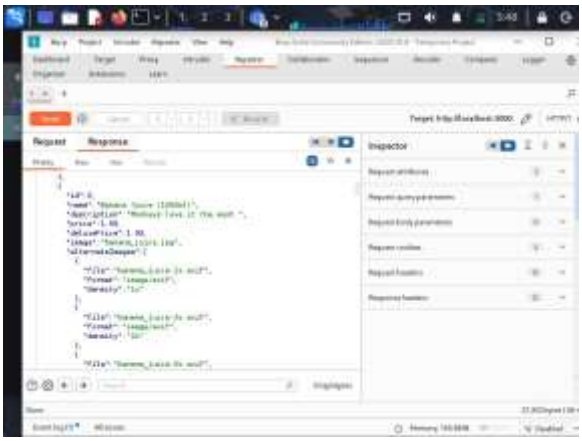


Figure: Product JSON response showing catalogue fields.

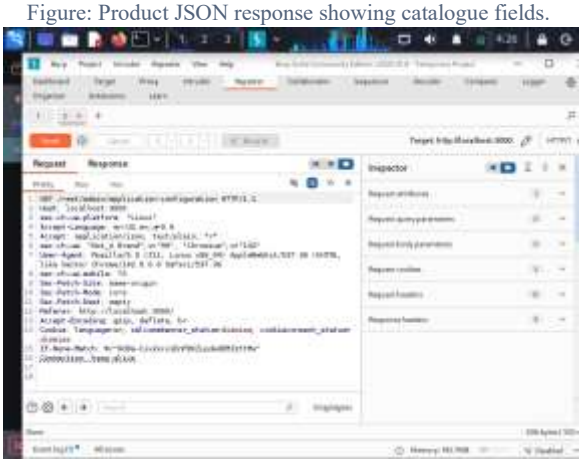


Figure: Administrative-looking endpoint request; response validation still required.

Conclusion

The supplied evidence demonstrates practical use of OWASP Juice Shop and Burp Suite to observe web traffic, inspect API requests, and review JSON responses. It provides a sound base for an advanced VAPT report. To claim a confirmed attack chain, add controlled before/after evidence for each weakness, verify the impact, and document the retest after remediation.

Evidence limitation: Screenshots are snapshots and may omit full request/response bodies. No successful account takeover, IDOR, or privilege escalation is asserted in this report without supporting proof.